

Android Security Assessment

OWASP UnCrackable App — Level 2

1. introduction

Target: UnCrackable-Level2.apk

Platform: Android

Analysis Type: Static Analysis, Smali Analysis, APK Modification

The assessment focused on:

- Identifying root detection mechanisms.
- Identifying debugger/debuggable detection.
- Identifying native anti-debugging & native password verification
- Recovering the hardcoded secret through static analysis.
- Modifying the application's Smali code to bypass root/debuggable checks.
- Modifying native anti debugging to bypass debuggable checks.
- Rebuilding and signing the modified APK.
- Installing and validating the modified APK in an Android 9 environment.

2. overview

2.1 environment setup

Test Environment: Android 9

Analysis Environment: Kali Linux

The Android application was analyzed inside an isolated Android 9 environment running through Virtual Master. The Kali Linux host communicated with the Android environment through ADB over TCP/IP.

2.2 tools

Tool	Purpose
JADX	Java/decompiled code analysis
Apktool	APK decoding, Smali analysis, rebuilding
apksigner	APK signing

ADB	Device communication and APK installation
Virtual Master	Android 9 test environment
Ghidra	Native code analysis

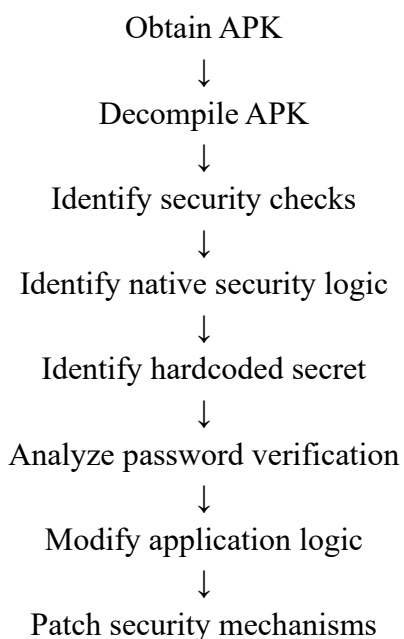
3. findings

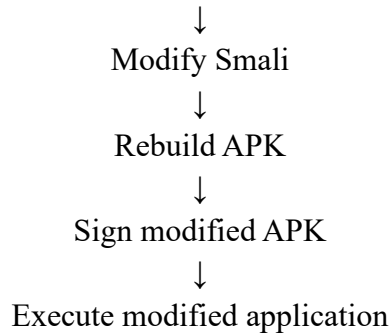
Finding	Severity	Result
Client-side root detection can be bypassed	Medium	Bypassed
Debuggable detection implemented entirely client-side	Low/Medium	Bypassed
Native anti debuggable detection	Low/Medium	Bypassed
Hardcoded secret in native code	High	Recovered
Native password verification	High	Successfully Bypassed

3.1 security impact

The primary security issue is that the application relies on client-side security controls and embedded cryptographic material.

An attacker with access to the APK can:





Therefore, security-sensitive secrets and trust decisions should not rely solely on client-side implementation.

4. Detail Findings

4.1 Initial Static Analysis

The original APK was analyzed using JADX.

The main application components identified were:

- MainActivity
- CodeCheck

Further analysis of the application revealed that several security mechanisms were implemented at the Java application layer, while the secret verification mechanism was delegated to a native library.

The application also loads a native library through:

```
static {  
    System.loadLibrary("foo");  
}
```

This indicates that the application depends on: libfoo.so

for part of its security-sensitive functionality. The initial static analysis therefore established that the application consists of both Java-side and native-side security mechanisms.

4.2 Root Detection Analysis

The application performs root detection during application initialization.

The relevant logic identified through JADX was:

```
if (b.a() || b.b() || b.c()) {
```

```
a("Root detected!");  
}
```

The application invokes multiple root-detection routines before continuing with the normal application flow.

Because these checks are implemented on the client side, the logic can be analyzed and modified by an analyst with access to the APK.

4.3 Debuggable Detection

The application also checks whether the application is running with the Android debuggable flag enabled.

The relevant implementation was identified through JADX:

```
if (a.a(getApplicationContext())) {  
    a("App is debuggable!");  
}
```

The underlying check uses the application's ApplicationInfo flags.

The purpose of this mechanism is to detect whether the APK was built with debugging enabled.

Because the check is performed locally, the resulting control flow can be modified through Smali analysis and patching.

4.4 Java Debugger Detection

Further analysis identified an additional debugger-detection mechanism implemented through an AsyncTask.

The application continuously checks:

```
Debug.isDebuggerConnected()
```

The reconstructed logic was:

```
while (!Debug.isDebuggerConnected()) {  
    SystemClock.sleep(100L);  
}
```

Once a debugger is detected, the application displays:

Debugger detected!

This mechanism is intended to interfere with dynamic debugging of the application.

However, because the mechanism exists within the application itself, it can be analyzed and modified by an analyst with access to the APK.

4.5 JNI Analysis

Further analysis of the Java code identified a native method:

```
private native boolean bar(byte[] bArr);
```

The Java wrapper method was:

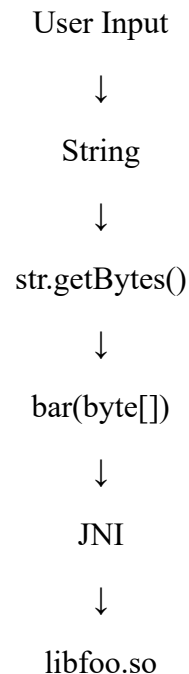
```
public boolean a(String str) {  
    return bar(str.getBytes());  
}
```

This indicates that the user-supplied string is converted into a byte array and passed to the native `bar()` method.

The application loads the native library using:

```
static {  
    System.loadLibrary("foo");  
}
```

Therefore, the Java-level verification process can be represented as:



This established that further analysis of the native library was required.

4.6 Native Binary Analysis

The native library: libfoo.so

was imported into Ghidra for further analysis.

The binary was identified as:

- ELF 64-bit LSB shared object
- ARM aarch64
- Dynamically linked
- Stripped

Ghidra was used to analyze:

- JNI functions
- Native control flow
- String references
- strcmp()
- fork()
- ptrace()
- waitpid()
- ARM64 instructions

The JNI implementation identified during the analysis was:

Java_sg_vantagepoint_uncrackable2_CodeCheck_bar

This function was particularly important because it connects the Java-side bar() method with the native password-validation logic.

4.7 Native Anti-Debugging Analysis

The native library contains an anti-debugging routine:

FUN_00100918()

The function uses several operating-system mechanisms, including:

- fork()
- getpid()
- ptrace()
- waitpid()

This mechanism attempts to interfere with external debugger attachment.

4.7.1 Native Anti-Debugging Patch

The function entry point originally contained: `sub sp, sp, #0x30`

The instruction was patched to: `ret`

As a result, the function immediately returns to its caller instead of executing the original anti-debugging logic. The modification therefore bypasses the native anti-debugging routine.

4.8 Secret Recovery

Further analysis of the native library revealed that the secret was embedded directly within the native binary.

During Ghidra analysis of: `Java_sg_vantagepoint_uncrackable2_CodeCheck_bar`

the following string was identified: Thanks for all the fish

The decompiled function contained: `builtin_strncpy(local_50, "Thanks for all the fish", 0x18);`

The function subsequently compares the supplied input against this value.

This demonstrates that the secret is not stored exclusively within the Java application layer but is instead embedded within the native library.

4.9 Native Password Verification Analysis

The Java layer exposes the native method: `private native boolean bar(byte[] bArr);`

The wrapper method:

```
public boolean a(String str) {  
    return bar(str.getBytes());  
}
```

passes the user input to the native library.

Further analysis of the native function showed that it:

- Obtains the input.
- Determines its length.

- Compares the length with 0x17.
- Performs strcmp().
- Returns 1 when the comparison succeeds.

The relevant native logic was reconstructed as:

```
if ((length == 0x17) &&
    (strcmp(input, secret, 0x17) == 0)) {
    return 1;
}
```

The recovered secret was: Thanks for all the fish

This demonstrates that the secret-validation mechanism remains entirely client-side, even though the validation logic has been moved into native code.

4.10 Rebuilding the Modified APK

After the required Smali and native modifications were completed, the APK was rebuilt using Apktool.

The rebuild command was: `apktool b apktool-output -o UnCrackable-Level2-unsigned.apk`

The build completed successfully and produced: Built apk into: UnCrackable-Level2-unsigned.apk

The resulting APK was unsigned.

4.11 APK Signing

A dedicated Android lab keystore was used to sign the modified APK.

The APK was signed using Android's apksigner:

```
apksigner sign \
    --ks android-lab.keystore \
    --ks-key-alias android-lab \
    --out UnCrackable-Level2-signed.apk \
    UnCrackable-Level2-unsigned.apk
```

The resulting APK was then ready for installation and dynamic validation.

4.12 Dynamic Validation

The signed APK was installed into the Android testing environment.

The modified application was then tested to verify the effectiveness of the modifications.

4.12.1 Test 1 — Root Detection

Expected before patch: Root detected!

After patch: Root detection bypassed

4.12.2 Test 2 — Debuggable Detection

Expected before patch: App is debuggable!

After patch: Debuggable detection bypassed

4.12.3 Test 3 — Incorrect Secret

Input: Incorrect value

Result: Nope...

4.12.4 Test 4 — Recovered Secret

Input: Thanks for all the fish

Result: Success!

5. Recommendation

For a production Android application:

- Do not rely on client-side root and debugger detection as a security boundary, root detection, debuggable checks, and anti-debugging mechanisms can be modified or bypassed by an attacker with access to the APK.
- Do not assume that moving security logic into native code makes it secure, JNI and native implementations can still be analyzed through native binary reverse engineering and tools such as Ghidra.
- Minimize sensitive information stored within the APK, any information distributed as part of the application should be assumed to be accessible to an attacker capable of obtaining and reverse engineering the APK.
- Apply code hardening and obfuscation, obfuscation and native code hardening can increase the cost of static analysis and reverse engineering, but these mechanisms should be

considered defense-in-depth measures rather than a replacement for secure application architecture.

6. Conclusion

The UnCrackable-Level2 application was successfully analyzed using static analysis, Smali analysis, APK modification, and dynamic testing. The assessment demonstrated that:

- Root detection mechanisms could be identified through static analysis and their client-side control flow could be bypassed.
- Debuggable application detection could be identified and bypassed through modification of the application's control flow.
- The application's native anti-debugging routine could be identified through analysis of libfoo.so using Ghidra.
- The hardcoded secret Thanks for all the fish could be recovered from the native library through static binary analysis.
- The modified APK could be rebuilt using Apktool and cryptographically signed using apksigner.
- The modified application could be installed and executed successfully in the Android 9 testing environment.
- The recovered secret could be used to successfully satisfy the application's native password verification mechanism.